CAPÍTULO 5

INSTRUCCIONES DE CONTROL DE FLUJO

Los programas que se han desarrollado hasta ahora se ejecutan en forma secuencial, esto es, el programa inicia su ejecución con la primera instrucción y continúa de arriba hacia abajo ejecutando cada una de las instrucciones hasta llegar a la última. En la mayoría de los casos, sin embargo, se requiere que el programa ejecute una serie de instrucciones dependiendo de sí una condición se cumple o de que ejecute una serie de instrucciones en forma repetitiva. Este tipo de programas se puede implementar mediante las instrucciones de control de flujo. Las instrucciones de control de flujo se clasifican en instrucciones de salto incondicional, salto condicional y repetitivas.

Instrucción De Salto Incondicional

La instrucción de salto incondicional hace que el control de flujo del programa salte a la instrucción cuya dirección está especificada por su operando.

jmp *dirección*

La instrucción **jmp** hace que la ejecución del programa continué en la dirección especificada por el operando.

Sintaxis:

jmp *etiqueta*
jmp *regW|memW*
jmp *memDW*

La dirección puede ser una etiqueta o estar en un registro o localidad de memoria. En el caso de usar un registro o localidad de memoria de una palabra, el salto es a una instrucción que se encuentra en el mismo segmento y el valor representa el desplazamiento de la instrucción con respecto al segmento. En el caso de que la dirección esté en una localidad de memoria de tipo palabra doble, el salto es a otro segmento y el valor es el segmento:desplazamiento de la instrucción.
La instrucción **jmp** no afecta a las banderas.

Instrucciones De Salto Condicional

Las instrucciones de salto condicional hacen que el control de flujo del programa salte a la instrucción cuya dirección está especificada por su operando si se cumple una condición dada.

***j**cond* *dirección*

Cambian la dirección de la siguiente instrucción a ser ejecutada dependiendo del valor de ciertas banderas .

Sintaxis:

j*cond* *etiqueta*

Todas las instrucciones de salto condicional operan en forma similar. **j***cond* es el mnemónico de la instrucción y *etiqueta* es la etiqueta de la dirección donde está la instrucción a la que va a saltar el programa. Los mnemónicos de las instrucciones de salto condicional pueden clasificarse en tres grupos: En la tabla 5-1a están los saltos cuya condición de salto es el resultado de una comparación de dos números no signados; en la tabla 5-1b se muestran los saltos cuya condición de salto es el resultado de una comparación de dos números signados; mientras que en la tabla 5-1c están los saltos cuya condición de salto es el resultado de una bandera o el registro **CX**. Aunque en las instrucciones de salto de las tablas 5-1a y 5-1b se dice que la condición de salto es el resultado de una comparación, la instrucción utiliza el valor de una o dos banderas para ejecutar o no el salto. Los valores de las banderas que producen el salto se muestran en la tercera columna en las tablas.

Tabla 5-1a. Saltos si la condición es una comparación no signada

Instrucción	Salta si...	Banderas
ja | **jnbe**	mayor | no menor o igual	**C** = 0 & **Z** = 0
jae | **jnb**	mayor o igual | no menor	**C** = 0
jb | **jnae**	menor | no mayor o igual	**C** = 1
jbe | **jna**	menor o igual | no mayor	**C** = 1 | **Z** = 1
je	igual	**Z** = 1
jne	diferente	**Z** = 0

Tabla 5.1b. Saltos si la condición es una comparación signada

Instrucción	Salta sí ...	Banderas
jg | **jnle**	mayor | no menor o igual	**S** = **O** & **Z** = 0
jge | **jnl**	mayor o igual | no menor	**S** = **O**
jl | **jnge**	menor | no mayor o igual	**S** ≠ **O**
jle | **jng**	menor o igual | no mayor	**S** ≠ **O** | **Z** = 1
je	igual	**Z** = 1
jne	diferente	**Z** = 0

Tabla 5.1c. Saltos si la condición es un estado de una bandera o el registro CX.

Instrucción	Salta sí ...	Banderas
jc	acarreo	**C** = 1
jo	sobreflujo	**O** = 1
jp | **jpe**	paridad | paridad par	**P** = 1
jpo | **jnp**	paridad non | no paridad	**P** = 0
js	signo	**S** = 1
jz	cero	**Z** = 1
jnc	no acarreo	**C** = 0
jno	no sobreflujo	**O** = 0
jns	no signo	**S** = 0
jnz	no cero	**Z** = 0
jcxz	si **CX** es cero	-

Algunos mnemónicos generan el mismo código máquina como **ja** y **jnbe** o **je** y **jz**.

Una restricción que tienen todos los saltos condicionales es de que la dirección a la que saltan debe de estar como máximo 128 bytes hacia atrás o 127 bytes hacia adelante del primer byte de la siguiente instrucción a la instrucción de salto. Por ejemplo, suponga que en el siguiente código, la instrucción a la que se desea saltar se encuentra más allá del rango de los -128 a 127 bytes:

```
        cmp     dx,1            ; if(DX == 1)
        je      error           ; goto error
        ...
error:                          ; Esta dirección está más
                                ; allá de los 127 bytes
```

El ensamblador nos desplegará el siguiente mensaje:

```
**Error** Relative jump out of range by n bytes
(**Error** Salto relativo fuera de rango por n bytes)
```

Una solución al problema anterior es recodificar el código anterior como:

```
        cmp     dx,1            ; if(DX != 1)
        jne     continua        ; goto continua
                                ; else
        jmp     error           ; goto error
continua:
        ...
error:                          ; Esta dirección está más
                                ; allá de los 127 bytes
```

Note que en este caso la condición de salto es la contraria a la del código anterior.

Ejemplos sobre instrucciones de salto

1. El siguiente programa encuentra el mayor de tres números signados almacenados en variables de tipo palabra.

```
;*********************************************************
; MAYOR3.ASM
;
; Este programa encuentra el mayor de tres datos signados
; almacenados en variables de una palabra. El pseudocódigo
; de este programa es:
;
;   AX = dato1
;   if(AX > dato2) goto sigcmp
;   AX = dato2
;
;sigcmp:
;   if(AX > dato3) goto fincmp
;   AX = dato3
;
;   mayor = AX
```

```
;*******************************************************
;****** CÓDIGO DE INICIO ***********************************

        ideal
        dosseg
        model   small
        stack   256

;****** VARIABLES DEL PROGRAMA *****************************

        dataseg
codsal  db      0
dato1   dw      ?
dato2   dw      ?
dato3   dw      ?
mayor   dw      ?

;****** CÓDIGO DEL PROGRAMA ********************************

        codeseg
inicio:
        mov     ax, @data       ; Inicializa el
        mov     ds, ax          ; segmento de datos

        mov     ax, [dato1]     ; AX = dato1
        cmp     ax, [dato2]     ; if(AX > dato2)
        jg      sigcmp          ;    goto sigcmp
        mov     ax, [dato2]     ; AX = dato2

sigcmp: cmp     ax, [dato3]     ; if(AX > dato3)
        jg      fincmp          ;    goto fincmp
        mov     ax, [dato3]     ; AX = dato3

fincmp: mov     [mayor], ax     ; mayor = AX

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN ******************************

        end     inicio
```

2. El siguiente programa suma los números enteros desde 1 hasta nfinal.

```
;*********************************************************
; SERIE1.ASM
;
; Este programa suma los números enteros de 1 hasta nfinal.
; El pseudocódigo de este programa es:
;
;   AX = 0
;   CX = nfinal
;
;   while(CX > 0)
;   {
;      AX += CX
;      CX–
;   }
;
;   suma = AX
;*********************************************************

;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256

;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
nfinal  dw      ?
suma    dw      ?

;****** CÓDIGO DEL PROGRAMA *******************************

        codeseg
inicio:
        mov     ax, @data       ; inicializa el
        mov     ds, ax          ; segmento de datos

        xor     ax, ax          ; AX = 0
        mov     cx, [nfinal]    ; CX = nfinal

while:  jcxz    endwhi          ; while(CX > 0)
                                ; {
        add     ax, cx          ;    AX += CX
        dec     cx              ;    CX--
        jmp     while           ; }

endwhi:
        mov     [suma], ax      ; suma = AX
```

```
salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN ****************************

        end     inicio
```

Ejercicios sobre instrucciones de salto

1. Cree un programa que obtenga el mayor de dos números signados del tipo palabra doble.

2. Cree un programa que multiplique dos variables tipo palabra no signadas. El resultado será una variable de tipo palabra doble. Utilice el algoritmo de sumas y corrimientos.

Instrucciones Repetitivas

El ensamblador del 8086 posee tres instrucciones especiales que permiten la construcción de ciclos.

loop *dirección*

Decrementa **CX** y luego salta si **CX** no es 0.

Sintaxis:

loop *etiqueta*

Esta instrucción decrementa el contenido del registro **CX** en uno, si el valor que queda en **CX** es diferente de cero, entonces la instrucción salta a la dirección especificada por *etiqueta*, la cual no debe encontrarse más allá de los 126 bytes hacia atrás o de los 127 bytes hacia adelante.

Esta instrucción se utiliza para crear ciclos que se repiten el número de veces especificado por el registro **CX**. Como la instrucción **loop** decrementa **CX** antes de probar si vale cero. Si **CX** vale cero al principio, al decrementar **CX** tomará el valor de 65535 y por lo tanto el ciclo ejecutará 65536 veces. Para prevenir esto preceda el ciclo con la instrucción **jcxz** como se muestra en el siguiente código.

```
        jcxz    enddo           ; if(CX == 0) goto enddo

do:                             ; do
                                ; {
        ...                     ;    instrucciones dentro
del
                                ;    ciclo
        loop    do              ; }
                                ; while(CX > 0)

enddo:
```

la instrucción **loop** no modifica las banderas.

loope | loopz *dirección*

Decrementa **CX** y luego salta si **CX** no es 0 y la bandera de cero **Z** vale 1.

Sintaxis:

loope | **loopz** *etiqueta*

Los mnemónicos **loope** y **loopz** representan la misma instrucción. Esta instrucción decrementa el contenido del registro **CX** en uno, si el valor que queda en **CX** es diferente de cero y la bandera de cero **Z** vale 1, presumiblemente puesta por una comparación previa, entonces la instrucción salta a la dirección especificada por *etiqueta*, la cual no debe encontrarse más allá de los 126 bytes hacia atrás o 127 bytes hacia adelante.

loopne | **loopnz** *dirección*

Decrementa **CX** y luego salta si **CX** no es 0 y la bandera de cero **Z** vale 0.

Sintaxis:

loopne | **loopnz** *etiqueta*

Los mnemónicos **loope** y **loopz** representan la misma instrucción. Esta instrucción decrementa el contenido del registro **CX** en uno, si el valor que queda en **CX** es diferente de cero y la bandera de cero **Z** vale 0, presumiblemente puesta por una comparación previa, entonces la instrucción salta a la dirección especificada por *etiqueta*, la cual no debe encontrarse más allá de los 126 bytes hacia atrás o 127 bytes hacia adelante.

Ejemplo sobre instrucciones repetitivas

El siguiente programa es una modificación del ejemplo 2 sobre las instrucciones de salto. Suma los números enteros desde 1 hasta nfinal.

```
;*************************************************************
; SERIE2.ASM
;
; Este programa suma los números enteros de 1 hasta nfinal.
; Esta versión utiliza la instrucción repetitiva loop. El
; pseudocódigo de este programa es:
;
;   AX = 0
;   CX = nfinal
;
;   if(CX == 0) goto endo
;
;   do
;   {
;      AX += CX
;   }
;   while(--CX > 0)

; enddo:
;   suma = AX
;*************************************************************

;****** CÓDIGO DE INICIO ************************************

        ideal
        dosseg
        model   small
        stack   256

;****** VARIABLES DEL PROGRAMA ******************************

        dataseg
codsal  db      0
nfinal  dw      ?
suma    dw      ?

;****** CÓDIGO DEL PROGRAMA *********************************

        codeseg
inicio:
        mov     ax, @data       ; Inicializa el
        mov     ds, ax          ; segmento de datos
```

```
        xor     ax, ax          ; AX = 0
        mov     cx, [nfinal]    ; CX = nfinal

        jcxz    endo            ; if(CX == 0) goto endo

do:                             ; do
                                ; {
        add     ax, cx          ;    AX += CX
        loop    do              ; }
                                  while(--CX > 0)
enddo:
        mov     [suma], ax      ; suma = AX

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN ********************************

        end     inicio
```

Ejercicio sobre instrucciones repetitivas

Modificar el ejercicio 3 sobre instrucciones lógicas para utilizar la instrucción repetitiva **loop**. El ejercicio 3 es hacer un programa que intercambie los bits de una variable de tipo palabra. El valor del bit 15 debe quedar en el bit 0, el valor del bit 14 debe quedar en el bit 1, etc. El resultado queda en la misma variable.

Bibliografía

1. Abel, Peter. Lenguaje Ensamblador y Programación para PC IBM y Compatibles. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1996.

2. Borland Int. Turbo Assembler Reference Guide. Version 1. Borland International. Scotts Valley, CA. 1988.

3. Brey, Barry B. Los microprocesadores Intel: 8086/8088, 80186, 80286, 80386 y 80486. Arquitectura, programación e interfaces. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1995.

4. Godfrey, J. Terry. Lenguaje Ensamblador para Microcomputadoras IBM para Principiantes y Avanzados. Prentice-Hall Hispanoamericana, S. A. México. 1991.

5. Hyde, Randall. The Art of Assembly Language Programming. Este libro se encuentra como una serie de documento PDF en el siguiente servidor FTP:

   ftp.cs.ucr.edu/pub/pc/ibmpcdir

6. Swan, Tom. Mastering Turbo Assembler. Hayden Books. Indiana, U.S.A. 1989.

Problemas

1. Un año bisiesto es aquel año que es divisible entre 4 pero no es divisible entre 100 a menos que sea divisible entre 400 en cuyo caso si es bisiesto. Crea un programa que determine si un año almacenado en la variable **anho** es un año bisiesto o no. La respuesta 0 para no y 1 para sí quedará en la variable **resul**. Llame este programa **TAREA501**.

2. La secuencia de Fibonacci, es la secuencia de enteros

   0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

   llamados números de Fibonacci. Podemos observar que a partir del tercer número de Fibonacci, estos están formados por la suma de los dos anteriores. El n-ésimo número de Fibonacci está dado por:

   Fib(n) = n sí n = 0, 1
   Fib(n) = Fib(n-1) + Fib(n-2) sí n >= 2

   hacer un programa que calcule el e-nesimo número de Fibonacci. El valor de n está en una variable de un byte llamada **n** y su Fibonacci quedará en la variable de tipo palabra **fibn**. Llame este programa **TAREA502**.

3. Hacer un programa que divida una variable de tipo palabra doble entre una variable de tipo palabra. Utilice el algoritmo de restas repetidas. El dividendo estará en la variable **dvdndo**, el divisor en la variable **divsor**, el cociente y el residuo en las variables de tipo palabra **cocien** y **resduo**. Llame este programa **TAREA503**.

4. Hacer un programa que encuentre el valor de n tal que:

   $1^2 + 2^2 + 3^2 + ... + n^2 \leq m$

El valor de m estará en la variable de tipo palabra doble **limite**. El valor de n quedará en la variable de tipo palabra **raiz.** El programa deberá llamarse **TAREA504**.

5. Haga un programa que separe los bits pares y nones de una variable de tipo palabra llamada **dato**. Los bits pares quedarán en una variable de un byte llamada **pares** y los bits nones en otra variable de un byte llamada **nones**. Utilice la instrucción repetitiva **loop**. El programa deberá llamarse **TAREA505**.

6. Haga un programa que establezca el bit de paridad par para una localidad de memoria de un byte llamada **dato**. Si la suma de unos de los bits 0 a 6 es par pondrá el bit 7 en cero y si la suma es non pondrá el bit 7 en uno. Utilice la instrucción repetitiva **loop**. El programa deberá llamarse **TAREA506**.